SPACL: A Combined Speculative Tableaux and Structural RDF/XML System for OWL Ontology Reasoning

Anusorn Chaikaew[a,1], Varin Chouvatut[a,2], Ekkarat Boonchieng[a,3,*]

[a]*Chiang Mai University, Department of Computer Science, Faculty of Science, Chiang Mai, 50200, Thailand*

Abstract

Expressive OWL reasoning is often constrained by combinatorial non-deterministic branch growth and high ontology ingest/materialization cost on large inputs. We present SPACL (Speculative Parallel Tableaux with Adaptive Conflict Learning), an open-source $\mathcal{ALC}/\mathcal{SHOIQ}$reasoner that combines branch-level speculative execution, reusable nogood information, and a structural RDF/XML processing pipeline for end-to-end ontology loading and consistency checking. The practical goal is to make expressive OWL reasoning usable under real deployment conditions.

SPACL uses a work-stealing speculative scheduler together with a conservative two-stage gate that keeps small or low-branch workloads on the sequential path. On synthetic disjunctive scalability workloads, this gate avoids severe always-parallel overhead and permits net speedup once branch structure becomes large enough, reaching up to 4.88× at the largest synthetic point evaluated.

These synthetic figures are supporting evidence only; the main claims of the paper come from repeated-run ontology benchmarks and stage-level parser analysis.

In the Standard Workload Suite (7 ontologies; 3 clean runs), per-ontology

*Corresponding author

Email addresses: `anusorn_chaikaew@cmu.ac.th` (Anusorn Chaikaew), `varin.ch@cmu.ac.th` (Varin Chouvatut), `ekkarat.boonchieng@cmu.ac.th` (Ekkarat Boonchieng)

[1]First author

[2]Second author

[3]Corresponding author

medians among successful comparable runs place SPACL lowest on 4/7 ontologies; ELK is lowest on the two deepest hierarchy files, and Pellet is lowest on `univ-bench.owl`. In the primary Large Head-to-Head benchmark (doid/go-basic/uberon; 3 clean runs), SPACL records the smallest median wall-clock values among successful comparable runs on all three tested ontologies under a common 900s timeout protocol. In repeated OWL2Bench core-profile validation (DL/EL/QL/RL), SPACL completes 12/12 runs and records the smallest median wall-clock values in this harness on the four tested profiles.

The largest end-to-end gains in the current evaluation come from parser and materialization optimization rather than from speculative reasoning alone. Stage-level A/B runs on DOID, GO-Basic, and UBERON show parse-only reductions of 96.96–97.98% relative to the previous structural baseline, and a supplementary ChEBI cold-text stress pair shows the same qualitative trend at larger scale. We therefore frame SPACL as a combined systems contribution. Speculative reasoning improves branch-heavy synthetic cases, while structural RDF/XML acceleration dominates the largest real-world wall-clock gains. All head-to-head results are reported as descriptive repeated-run wall-clock comparisons under a fixed harness, not as formal significance tests.

Keywords: $\mathcal{ALC}/\mathcal{SHOIQ}$, Tableaux Reasoning, Parallel Algorithms, Nogood Learning, Description Logics, Semantic Web

---

1. Introduction

Description Logics (DLs) provide the formal basis of the Semantic Web, and OWL2 DL is the corresponding W3C ontology language standard [1]. Tableaux methods remain important for expressive OWL reasoning because they support sound and complete model construction for rich TBox and ABox tasks [2]. In practice, however, expressive reasoning is often limited by the combinatorial growth of non-deterministic branches and by end-to-end loading costs on large ontologies.

This paper studies that bottleneck through an implemented $\mathcal{ALC}/\mathcal{SHOIQ}$reasoning path. We focus on two practical questions. First, can speculative parallel tableaux reduce branch-exploration cost without paying unacceptable overhead on easy workloads? Second, when large RDF/XML ontologies dominate runtime, how much of the observed end-to-end behavior is actually due to parsing and materialization rather than to reasoning proper? These questions matter both for ontology-engineering workflows and for our longer-term goal of using reasoning

as a validation component in distributed application settings. As shown later by our stage-level A/B results, the dominant bottleneck in the evaluated real-world RDF/XML ontologies lies in ingest/materialization rather than in branch exploration; speculative scheduling remains useful, but its clearest impact appears on branch-heavy synthetic and inconsistent workloads rather than as the primary source of the largest biomedical wall-clock gains.

1.1. Research Questions

The paper addresses three questions.

- **RQ1.** Can speculative parallel tableaux with conflict-driven nogood learning improve OWL consistency-checking performance?
- **RQ2.** Can adaptive parallelism reduce overhead on small workloads while preserving scalability on larger workloads?
- **RQ3.** For large RDF/XML ontologies, which stages dominate end-to-end runtime, and how much can structural parsing/materialization optimization reduce that bottleneck while preserving implementation-aligned semantic behavior?

1.2. Contributions

SPACL contributes four elements. First, it combines branch-level speculative work stealing with reusable nogood information in a single open-source implementation (Sections 4.1 and 4.2). Second, it uses a conservative adaptive gate that keeps low-branch workloads on the sequential path while allowing larger speculative workloads to use multiple workers (Section 4.3). Third, it introduces a structural RDF/XML pipeline that reuses the same semantic handlers as the baseline path while substantially reducing parse/materialization cost on large biomedical ontologies (Section 5.2). Fourth, it evaluates the combined system with repeated-run synthetic, stage-level, and cross-reasoner benchmarks under a reproducible Docker-containerized protocol (Sections 6.2 and 6.7). In the current evaluation, the clearest practical headline result is parser/materialization improvement on large RDF/XML ontologies, while speculative-reasoning impact is most visible on branch-heavy synthetic and inconsistent workloads.

2. Related Work

Tableau and hypertableau reasoners such as Pellet, Openllet, HermiT, and Konclude show that expressive OWL reasoning remains viable when branching, blocking, and caching are carefully engineered [3, 4, 5, 6, 7]. ORE benchmark campaigns also provide a broader comparative backdrop for these systems, especially for Konclude and other mature OWL reasoners [8]. At the other end of the design space, ELK demonstrates the throughput advantage of profile-restricted consequence-based reasoning [9]. This gives the central tension for our work: expressive reasoning remains useful, but branching and operational overhead often dominate runtime.

Parallel reasoning research addresses that tension through shared-memory classification frameworks, work-stealing schedulers, parallel ABox reasoning, and distributed materialization systems [10, 11, 12, 13, 14, 15, 16]. These studies show that parallelism helps only when coordination cost is controlled. Conflict-driven learning offers another lever. In SAT/CSP it is standard; in DL systems, related reuse ideas appear in hypertableau optimizations and tableau–saturation hybrids [17, 18, 7, 19, 20]. SPACL is positioned as a practical open-source combination of these strands for the implemented $\mathcal{ALC}/\mathcal{SHOIQ}$handling path exercised here.

3. Formal Framework

SPACL follows a tableau-style reasoning path for the implemented $\mathcal{ALC}/\mathcal{SHOIQ}$fragment exercised in this paper. Conjunction expands in place, existential and universal restrictions propagate as usual, and disjunction creates branch points. The main design choice is operational rather than axiomatic: how to schedule those branches, when to parallelize them, and when to prune them safely.

Speculative execution in SPACL is a schedule change, not a rule change. The implementation may explore selected disjunctive branches concurrently, but every branch is still evaluated by the same underlying rule-handling path. Learned nogoods are admitted only after contradiction is rechecked, so pruning is used only for branch contexts already verified unsatisfiable.

3.1. Tableaux Basis

For reference, the implemented path follows the familiar $\mathcal{ALC}$ concept grammar

$$C, D ::= A \mid \top \mid \bot \mid \neg C \mid C \sqcap D \mid C \sqcup D \mid \exists r.C \mid \forall r.C, \tag{1}$$

with the higher-expressivity handling used in the implementation treated here as an engineering extension of this base presentation. Consistency checking proceeds by constructing a completion graph $\mathcal{G} = (V, E, \mathcal{L})$, where nodes represent individuals, role-labeled edges represent relationships, and $\mathcal{L}$ maps each node to its active concept labels.

A clash occurs when a node simultaneously contains C and $\neg C$ for some concept C, or contains $\bot$. The sequential tableaux path expands conjunctions in place, creates new branch states for disjunctions, adds witnesses for existential restrictions, and propagates universal restrictions across matching role edges. Table 1 summarizes these core $\mathcal{ALC}$ rules only to anchor the operational discussion that follows.

Table 1: Compact $\mathcal{ALC}$ Expansion Rules

Rule	**Premise**	**Action**
$(\sqcap)$	$x : C \sqcap D \in \mathcal{L}(x)$	add C, D to $\mathcal{L}(x)$
$(\sqcup)$	$x : C \sqcup D \in \mathcal{L}(x)$	branch with C or D added to $\mathcal{L}(x)$
$(\exists)$	$x : \exists r.C \in \mathcal{L}(x)$	create y with $(x, y) : r$ and $C \in \mathcal{L}(y)$
$(\forall)$	$x : \forall r.C \in \mathcal{L}(x)$, $(x, y) : r$	add C to $\mathcal{L}(y)$

3.2. Adaptive Threshold

This subsection specifies the implemented gate as an operational heuristic, not as a predictive runtime theory. Parallel mode is enabled only when both a branch-count estimate and a lightweight cost score exceed conservative thresholds. The branch estimate is

$$B(\mathcal{O}) = \max\left(1,\ |\mathcal{D}| + 2 \times |\text{DisjointAxioms}| + \frac{|\text{Classes}|}{10}\right), \tag{2}$$

where $|\mathcal{D}|$ is the number of detected disjunctions. Intuitively, $|\mathcal{D}|$ captures explicit branch points, the disjoint-axiom term captures contradiction pressure that can trigger branch checks, and $\frac{|\text{Classes}|}{10}$ is a coarse size stabilizer so large ontologies with sparse disjunctions are not always treated as trivial. Expression-level scoring uses

implementation weights

$$\text{cost}(\text{ObjectUnionOf}(ops)) = 50 \times |ops| + \sum \text{cost}(op), \tag{3}$$

$$\text{cost}(\text{ObjectIntersectionOf}(ops)) = 25 \times |ops| + \sum \text{cost}(op), \tag{4}$$

$$\text{cost}(\text{ObjectComplementOf}(e)) = 15 + \text{cost}(e), \tag{5}$$

$$\text{cost}(\text{atomic}) = 10. \tag{6}$$

The prototype enables parallel mode only when $B(\mathcal{O}) \geq \tau$ and the accumulated detected-disjunction cost exceeds $\theta \cdot p$, with $\tau = 2000$, $\theta = 500\mu s$, and p the worker count. The weights are calibrated implementation parameters from pilot sweeps over synthetic and warm-up ontology traces; they are not description-logic complexity constants.

Heuristic crossover rule. For estimated disjunction cost D, parallel overhead o_{par}, worker count p, and an empirical nogood-hit proxy α, a useful operating inequality is

$$D > \frac{p \cdot o_{par}}{1 - \alpha}. \tag{7}$$

We use this inequality only as an intuition aid for threshold tuning: it explains why speculative execution should stay disabled when coordination cost dominates the expected disjunctive work. The equations in this subsection are implementation-facing heuristics for gating and operating-region selection; they are not fitted predictive runtime models and are included chiefly to make the implemented decision policy explicit and reproducible.

3.3. Nogood Learning

The pruning mechanism can be stated compactly. In this paper, a *nogood* is a reusable set of branch constraints that is verified to produce contradiction and can therefore be used to prune later branch contexts. Let Γ_b denote the active branch constraints for a contradictory branch b. A candidate nogood is a subset $\mathcal{N} \subseteq \Gamma_b$ such that

$$\mathcal{O} \cup \mathcal{N} \models \bot. \tag{8}$$

Pruning is authorized only after revalidation on the same checking path used by the implementation, and only when the learned core subsumes the current branch context:

$$\mathcal{N} \subseteq \Gamma_b \quad \wedge \quad \mathcal{O} \cup \mathcal{N} \models \bot. \tag{9}$$

This is intentionally conservative. SPACL does not treat every contradictory trace as globally reusable; it only admits verified branch-level cores that remain contradiction-preserving under the current ontology and constraint context. Algorithm 1 summarizes the pruning-safe extraction procedure used to validate such cores before they are admitted for reuse.

Algorithm 1 Verified Nogood Extraction (Pruning-Safe)

Require: Contradictory branch constraints Γ_b, ontology $\mathcal{O}$
Ensure: Verified nogood $\mathcal{N}$ for pruning, or $\emptyset$

```
N ← Γ_b
for each constraint expression e ∈ Γ_b do
    if O ∪ (N \ {e}) ⊨ ⊥ then
        N ← N \ {e}
    end if
end for
if O ∪ N ⊨ ⊥ then
    return N
else
    return ∅
end if
```

3.4. Correctness Scope

Our correctness claim is intentionally limited. Under an idealized model with fair exhaustive exploration of generated disjunctive branches and a complete sequential tableaux reference procedure for the same implemented $\mathcal{ALC}/\mathcal{SHOIQ}$handling path, speculative scheduling preserves soundness and transfers conditional completeness: reordering branches does not invalidate a satisfiable branch, and verified nogoods prune only branch contexts already checked unsatisfiable. The prototype evaluated here adds engineering bounds such as selected-disjunction scope, bounded speculative depth, and timeout limits, so the results in Section 6 are empirical evidence for the implementation rather than a full completeness proof for unrestricted SHOIQ reasoning.

4. The SPACL Algorithm

Figure 1 summarizes the execution flow used in this revision. The key point is that SPACL is a pipeline: text ingestion and materialization produce the internal

ontology representation, the adaptive gate decides whether speculative work generation is justified, and only then does speculative branch exploration with nogood reuse proceed. At branch level, SPACL still relies on the same underlying sequential tableaux rule-handling path; the contribution lies in how work is scheduled, bounded, and pruned around that baseline path rather than in replacing the local rule application semantics themselves.

SPACL execution flow

1. Parse and materialize ontology into the internal representation using the baseline semantic handlers.
2. Estimate branch count and detected-disjunction cost.
3. If the adaptive gate is closed, run the sequential tableaux path.
4. If the adaptive gate is open, generate bounded branch-constrained work items.
5. Explore work items with a work-stealing scheduler.
6. When contradiction is reached, validate and cache a nogood before pruning.
7. Return satisfiable/unsatisfiable/timeout from the first decisive outcome.

Figure 1: Compact execution-flow summary of the combined SPACL system.

Figure 2 complements this execution summary with the runtime architecture used in the implementation. The intent is to make the interaction between the parser/materialization path, adaptive controller, speculative workers, and nogood store explicit without reintroducing a longer architectural walkthrough.

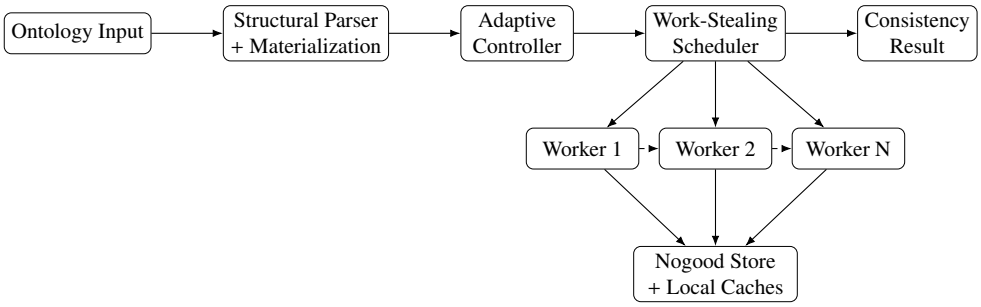

Figure 2: Compact runtime architecture of SPACL. Solid arrows denote main control/data flow; dashed arrows denote worker coordination.

Algorithm 2 gives the compact control flow of the reasoning path used in this revision.

Algorithm 2 SPACL Consistency Check

Require: Ontology O, configuration C
Ensure: **true** if consistent, **false** if inconsistent, otherwise timeout/unknown

```
Parse and materialize O into the internal ontology representation
b ← EstimateBranchCount(O)
d ← EstimateDetectedDisjunctionCost(O)
if b < C.τ or d < C.θ · C.workers then
    return SequentialConsistencyCheck(O)
end if
Q ← bounded branch-constrained work items from selected disjunctions
start worker pool with shared nogood store and thread-local caches
while no decisive result has been returned do
    workers pop or steal work items, expand branches, and validate nogoods
    before pruning
    if a worker finds a clash-free branch then
        return true
    else if all reachable work items are closed by clash or verified pruning
    then
        return false
    end if
end while
return timeout/unknown
```

4.1. Speculative Work-Stealing

When the adaptive gate is open, SPACL creates branch-constrained work items from selected disjunctions and distributes them through a work-stealing scheduler [12]. Each worker keeps a local deque and steals from other workers only when idle, which avoids a centralized queue on the hot path. The current prototype bounds speculative depth and selected-disjunction scope so that branch creation remains measurable and recoverable under the benchmark harness.

4.2. Nogood Learning and Caching

When a speculative branch reaches contradiction, the implementation attempts to learn a branch-level nogood from the active branch constraints. A candidate core is greedily minimized and admitted only after contradiction is revalidated by the same checking path used for pruning. Concretely, the candidate core is replayed through the same contradiction checker used on the live branch state, and it is admitted only if contradiction is reproduced and the core remains a subset of

the active branch constraints; otherwise it is discarded. Workers consult thread-local nogood caches before synchronizing with the global store, which keeps most repeated checks off the shared hot path in the inconsistent-workload ablations reported later.

4.3. Adaptive Threshold

The implemented branch estimate is given in Equation 10:

$$\text{branches} = \max\left(1,\ |\text{DetectedDisjunctions}| + 2 \times |\text{DisjointAxioms}| + \frac{|\text{Classes}|}{10}\right). \tag{10}$$

The divisor 10 keeps ontology-size contribution on the same order as the disjunction signal in pilot sweeps, so low-branch taxonomies do not trigger speculative execution prematurely. Calibration was done by replaying synthetic union/complement controls together with warm-up DOID and GO-Basic traces under candidate (τ, θ) pairs, then selecting a conservative operating point that suppresses low-branch overhead while still allowing activation on branch-heavy controls. These sweeps showed that the robust operating region begins at $\tau \geq 2000$; the implementation therefore keeps a conservative default at $\tau = 2000$.

5. Implementation

5.1. System Overview

SPACL is implemented in Rust 1.84.0. The main reasoner maintains a shared ontology representation, a global worker pool, and thread-local nogood caches backed by a bounded shared store. Rust and Crossbeam are used here for engineering reasons rather than as claims in themselves: the goal is to keep speculative scheduling and shared-state coordination explicit while avoiding garbage-collection pauses and coarse-grained locking on the hot path [21].

The public implementation includes parsers for the syntaxes exercised in this study, with RDF/XML being the dominant path in the large-ontology experiments. Full OWL2 DL conformance across every syntax variant is outside the scope of this revision; the results reported below concern the implemented $\mathcal{ALC}/\mathcal{SHOIQ}$handling path exercised by the evaluated workloads.

5.2. Structural RDF/XML Parser and Materialization

The large-ontology path uses a two-phase structural RDF/XML pipeline. Phase A streams triples and interns subject/predicate/object terms to dense IDs; Phase

B resolves those IDs through dense caches and dispatches each triple through the same semantic handler core as the baseline parser. The objective is not to change reasoning semantics but to reduce hot-path term-resolution and materialization cost on large RDF/XML files. The stage-level A/B results in Section 6.5 show that this parser/materialization path, rather than speculative reasoning itself, is the dominant source of end-to-end gains on the evaluated biomedical ontologies.

Phase A (Ingest + Intern) streams RDF/XML triples, interns subject/predicate/object terms to dense integer IDs, and stores compact records $\langle s, p, o \rangle$ in arrival order. **Phase B (Materialize)** resolves those IDs through dense caches and subject-cursor fast paths, computes predicate tags, and then dispatches through `apply_triple_terms_core`, preserving the same semantic handler path as the baseline parser.

We model end-to-end parser load time with Equation 11:

$$T_{\text{total}} = T_{\text{ingest}} + T_{\text{materialize}} + T_{\text{io}}. \tag{11}$$

For a cache layer $k \in \{\text{subj}, \text{pred}, \text{obj}\}$, Equation 12 defines the hit rate and expected lookup time:

$$H_k = \frac{\text{hits}_k}{\text{hits}_k + \text{misses}_k}, \quad E[t_k] = H_k \cdot t_{hit,k} + (1 - H_k) \cdot t_{miss,k}. \tag{12}$$

Replacing repeated hash-map resolution with dense-ID vector caches reduces both $t_{hit,k}$ and $t_{miss,k}$ on the hot path.

To relate parser acceleration to overall wall-clock improvement, Equation 13 applies Amdahl's law:

$$S_{\text{e2e}} = \frac{1}{(1 - f_{\text{parse}}) + \frac{f_{\text{parse}}}{S_{\text{parse}}}}, \tag{13}$$

where f_{parse} is the pre-optimization parse/materialization fraction and S_{parse} is the parser-stage speedup. These equations are explanatory bookkeeping relations for interpreting the measured stage shifts in Section 6.5; they are not presented as a fitted predictive ingest model. Algorithm 3 then summarizes the two-phase handler-equivalent ingest/materialization path used in the implementation.

6. Evaluation

6.1. Experimental Setup

Hardware: Intel Xeon Silver 4214 (12 cores/24 threads, 2.2GHz base) with 64GB DDR4-2400 RAM, Ubuntu 22.04 LTS.

Algorithm 3 Structural RDF/XML Parse with Handler-Equivalent Materialization

Require: RDF/XML stream X

Ensure: Ontology $\mathcal{O}$ targeting handler-equivalent behavior to the baseline parser on the implemented path

Initialize interner I, record buffer R, ontology $\mathcal{O}$
for each triple t emitted by the streaming RDF/XML parser **do**
  $s \leftarrow \textsc{InternSubject}(I, t.subject)$
  $p \leftarrow \textsc{InternPredicate}(I, t.predicate)$
  $o \leftarrow \textsc{InternObject}(I, t.object)$
  Append (s, p, o) to R
end for
Initialize dense caches $(C_{subj}, C_{pred}, C_{obj})$
for each record (s, p, o) in R **do**
  $\hat{s} \leftarrow \textsc{ResolveSubject}(s, C_{subj})$
  $\hat{p} \leftarrow \textsc{ResolvePredicate}(p, C_{pred})$
  $\hat{o} \leftarrow \textsc{ResolveObject}(o, C_{obj})$
  $\rho \leftarrow \textsc{PredicateTag}(\hat{p})$
  $\textsc{ApplyTripleTermsCore}(\hat{s}, \rho, \hat{o}, \mathcal{O})$
end for
return $\mathcal{O}$

Software: Rust 1.84.0, release mode with `-C opt-level=3 -C lto=fat`.
Benchmarks: Criterion.rs synthetic microbenchmarks, plus Docker-containerized repeated-run ontology benchmarks for Standard, Large, and OWL2Bench core suites (3 clean runs each).

Workloads: synthetic hierarchies (100–100,000 classes), synthetic disjunctive tests, `univ-bench.owl`, large biomedical ontologies (DOID, GO-Basic, UBERON), supplementary PATO and ChEBI cases, and OWL2Bench core profiles (DL/EL/QL/RL). **Ontology releases used in this run**: DOID `2026-02-02` (`doid/releases/2026-02-02/doid.owl`); GO-Basic `2026-01-23` (`go/releases/2026-01-23/go.ow` UBERON `2025-12-04` (`uberon/releases/2025-12-04/uberon.owl`); PATO `2025-05-14` (`pato/releases/2025-05-14/pato.owl`); ChEBI release `248` with header date string `06:01:2026 14:33` (as encoded in `chebi.owl` metadata).

6.2. Benchmark Protocol

Primary cross-reasoner comparison uses SPACL, HermiT, Openllet, ELK, JFact, and Pellet. Standard and OWL2Bench suites use a 300s timeout; the large biomedical suite uses 900s. Tables report median wall-clock time over three clean

runs per ontology/reasoner pair. To expose run-to-run spread without overclaiming inferential statistics, we additionally report P95 wall time where space permits. These values are descriptive deployment-level measurements (startup + parsing + reasoning), not formal significance tests or parser-neutral kernel timings.

For reproducibility, benchmark orchestration is provided in the public repository scripts (`benchmarks/competitors/scripts/run_benchmarks.sh`, `benchmarks/competitors/scripts/run_stage_benchmark.sh`, and `benchmarks/competitors/scripts/run_stage_suite.sh`). Konclude is reported separately because it requires a preprocessing-enabled external-binary path and a separate rerun campaign.

6.3. Scalability Results

Table 2 summarizes the synthetic scalability results. These values are supporting evidence for the adaptive parallel design rather than the primary basis of the paper's real-world claims.

Table 2: Synthetic Scalability Results

Classes	**Sequential (μs)**	**SPACL (μs)**	**Speedup**	**Throughput (M exp./s)**
100	13.3	20.9	0.64×	4.8
500	75.9	84.3	0.90×	5.9
1,000	159.7	158.4	1.01×	6.3
5,000	805.9	277.0	**2.91**×	18.1
10,000	1,865.3	382.3	**4.88**×	**26.2**

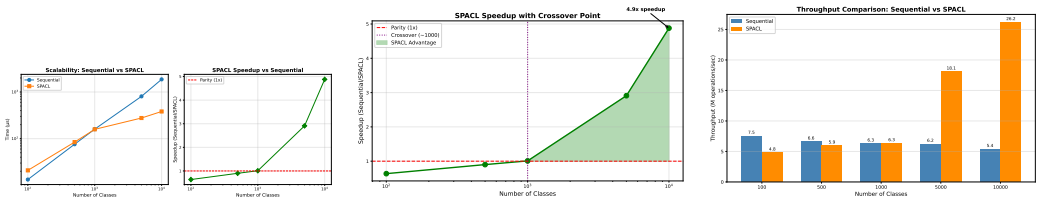

Figure 3: Synthetic scalability visual summary corresponding to Table 2: runtime trend (left), speedup over the sequential baseline (center), and throughput (right).

The crossover appears around 1,000 classes, and Figure 3 shows the same transition visually: below that point, speculative overhead largely cancels any gain; above it, the branch structure is large enough for parallelism to amortize coordination cost. Throughput here denotes synthetic expansion operations per second in the microbenchmark harness. The right panel compares both sequential

and SPACL throughput, while the throughput column in Table 2 reports the SPACL throughput values used for compact tabular comparison. This is consistent with the conservative adaptive-gating design and explains why the main real-world evidence is separated from the synthetic scalability signal.

6.4. Adaptive Threshold Behavior

The adaptive gate is intended as an overhead-control mechanism rather than as the primary explanation for the large biomedical gains. Its role is to avoid obviously bad parallel activation on low-branch workloads, not to serve as a general runtime predictor. Table 3 summarizes representative small disjunctive cases comparing forced sequential execution, the adaptive policy, and a naive always-parallel baseline.

Table 3: Adaptive Threshold Performance on Small Disjunctive Cases

Test Case	**Sequential**	**Adaptive**	**Always-Parallel**	**Adaptive/Sequential**	**AP/Adaptive**
Simple (1 union)	9.0μs	14.5μs	1,069μs	1.61×	**73.5**×
Small (2 unions)	7.9μs	24.7μs	1,141μs	3.13×	**46.1**×
Medium (5 unions)	9.1μs	58.9μs	1,116μs	6.47×	**18.9**×

The adaptive policy remains slightly slower than pure sequential execution on trivial cases, but it is still far better than forcing parallel execution everywhere. We therefore interpret the gate conservatively: it is justified as a safe operating policy for branch-heavy workloads, not as a claim that the chosen coefficients are uniquely optimal. Calibration sweeps used in this revision showed that the robust region begins at branch threshold
tau
*ge*2000; in the warm-up biomedical traces used for calibration, DOID and GO-Basic stayed on the sequential path because no speculative disjunctions were detected.

To expose scheduler policy more directly under controlled conditions, we also ran a fixed synthetic workload (`union_4x10`) with deliberately lowered gate settings ($\tau = 4$, $\theta = 1\mu s$) so that all speculative modes would activate on the same branch-heavy instance. Table 4 reports the resulting medians. This is a policy-isolation experiment rather than a default-setting benchmark: its role is to show that, once speculative execution is forced into play on the same workload, the adaptive policy materially improves over naive always-parallel scheduling. In this experiment, `nogood_hits` remained zero, so we treat it as scheduler evidence only, not as positive nogood-learning evidence.

Table 4: Controlled Scheduler Policy Ablation on `union_4x10` (median wall-clock over 3 runs; gate overrides $\tau = 4$, $\theta = 1\mu s$ used only to expose policy behavior)

Mode	**Median (ms)**	**Used Parallel**
Sequential	0.146	No
Adaptive	53.149	Yes
Adaptive, no nogood	84.560	Yes
Always-parallel	108.012	Yes

To verify that branch-level pruning can activate in the current implementation, we ran a second controlled synthetic stress case (`reused_conflict_12`) with gate overrides ($\tau = 8$, $\theta = 1\mu s$). Table 5 summarizes this run. Adaptive mode showed non-zero pruning in all repeats (hits/prunes of 418, 4159, and 13), while the no-learning and always-parallel variants stayed at zero pruning. Runtime spread is still high, so we report this as mechanism-activation evidence rather than as a stable production-level speedup claim.

Table 5: Controlled Nogood-Activation Ablation on `reused_conflict_12` (median wall-clock over 3 runs; gate overrides $\tau = 8$, $\theta = 1\mu s$)

Mode	**Median (ms)**	**Median nogood hits**
Sequential	0.452	0
Adaptive	1037.632	418
Adaptive, no nogood	6821.705	0
Always-parallel	14424.697	0

6.5. Real-World Ontology Evaluation

To isolate parser/materialization effects, we performed controlled stage-level A/B runs before and after the structural RDF/XML pipeline changes on large biomedical ontologies drawn from widely used OBO/BioPortal-style resources [22]. Table 6 reports the primary biomedical pairs together with a supplementary ChEBI stress case under the revised cold-text contract.

Across the three primary biomedical ontologies, parse/materialization was the dominant cost in the original path. After the structural pipeline changes, the reasoning stage stayed comparatively small in these traces, so the main end-to-end gain in this revision is parser-driven. The ChEBI stress pair is reported only as supplementary evidence, but it shows the same qualitative pattern at larger scale.

Table 6: Stage-Level A/B on Real-World RDF/XML Ontologies (cold `.owl` runs; E2E = end-to-end wall clock; ChEBI is supplementary)

Ontology	Parse Old	Parse New	Parse Red.	E2E Old	E2E New	E2E Red.
DOID	27307	830	96.96%	30502	4073	86.65%
GO_Basic	158306	3472	97.81%	162519	7409	95.44%
UBERON	145843	2952	97.98%	149850	6632	95.57%
ChEBI (supp.)	970388	23740	97.55%	985466	36472	96.30%

6.6. Comparison Context

The comparison set mixes engines with different implementation languages and, in ELK's case, a narrower logic profile than the full $\mathcal{ALC}/\mathcal{SHOIQ}$path exercised by SPACL. Table 7 summarizes this context so that wall-clock rankings are not overinterpreted as expressivity-equivalent wins.

Table 7: Feature Comparison with Representative Reasoners

Reasoner	Lang	Parallel	Learning	Source	Profile
SPACL	Rust	yes	yes	open	$\mathcal{ALC}/\mathcal{SHOIQ}$
Konclude	C++	yes	no	external binary	SROIQ
Pellet	Java	no	no	open	SROIQ
Openllet	Java	no	no	open	SROIQ
HermiT	Java	no	no	open	SROIQ
ELK	Java	yes	no	open	EL++

6.7. Head-to-Head Performance Comparison

6.7.1. Standard Workload Suite

Table 8 reports median wall-clock results for the Standard Workload Suite.

Table 8: Standard Workload Suite Consistency Benchmark (median wall time across 3 clean runs, ms; timeout 300s; SPACL P95 shown for spread)

Ontology	SPACL	HermiT	Openllet	ELK	JFact	Pellet	SPACL P95
disjunctive_simple.owl	**2273**	3446	3383	3718	3466	3310	2337
disjunctive_test.owl	**2113**	3323	3007	2979	3362	2659	2318
hierarchy_100.owl	**2096**	3391	3070	2910	3271	2838	2384
hierarchy_1000.owl	**2190**	3498	3220	3055	4110	3222	2330
hierarchy_10000.owl	8007	4131	4084	**3579**	FAIL	3644	8290
hierarchy_100000.owl	40210	7706	8881	**5816**	FAIL	10867	40273
univ-bench.owl	2951	3345	2930	2866	3196	**2742**	3096

Legend: FAIL = runtime failure in this setup; bold = lowest median wall time among successful comparable runs per ontology.

This suite is best read as a deployment sanity check rather than as the strongest evidence for reasoning quality, because startup and parser costs are still visible at this scale. We therefore retain it mainly to show that the implementation behaves competitively under the benchmark harness; all systems except JFact complete every Standard-suite case under the 300s policy. SPACL run-to-run spread in this suite is modest in relative terms (P95/median about 1.00–1.14× from the reported repeated runs).

6.7.2. Large Head-to-Head

Table 9 reports the primary biomedical head-to-head results.

Table 9: Large Head-to-Head Consistency Benchmark (median wall time across 3 clean runs, ms; timeout 900s; SPACL P95 shown for spread)

Ontology	**SPACL**	**HermiT**	**Openllet**	**ELK**	**JFact**	**Pellet**	**SPACL P95**
doid.owl	**3447**	20295	14266	13406	60443	28771	3701
go-basic.owl	**7329**	17616	17747	15976	41847	83238	8000
uberon.owl	**6677**	110560	TO	15594	FAIL	TO	7363

Legend: TO = timeout; FAIL = runtime failure in this setup; bold = lowest median wall time among successful comparable runs per ontology.

Among successful comparable runs, SPACL yields the smallest median wall-clock values on all three ontologies in the primary large suite. SPACL, HermiT, and ELK complete every run under the 900s policy; Openllet and Pellet time out on UBERON, and JFact fails on UBERON in this harness. SPACL P95/median remains close (about 1.07–1.10×), indicating moderate run-to-run spread in this campaign. Accordingly, the '6/9' success counts in Table 10 are not dispersed failures: they reflect complete success on DOID and GO-Basic (6 runs total) together with complete failure or timeout on the larger UBERON ontology (3 runs).

6.7.3. Failure-Aware Suite Summary

Success-only per-ontology medians do not capture timeout penalties directly. We therefore also report a capped expected wall-time (CEW) summary over the primary comparison set in Table 10, where timeout or failure is replaced by the suite timeout before averaging.

Table 10: Suite-Level Failure-Aware CEW Summary (3 clean runs; lower is better)

Suite	Reasoner	Comparable Cases	Success / Comparable	CEW (ms)
Standard	ELK	21/21	21/21 (100%)	**3578.1**
Standard	Openllet	21/21	21/21 (100%)	4120.9
Standard	HermiT	21/21	21/21 (100%)	4159.8
Standard	Pellet	21/21	21/21 (100%)	4204.0
Standard	SPACL	21/21	21/21 (100%)	8516.8
Standard	JFact	21/21	15/21 (71.4%)	88239.5
Large	SPACL	9/9	9/9 (100%)	**5948.4**
Large	ELK	9/9	9/9 (100%)	15084.2
Large	HermiT	9/9	9/9 (100%)	50203.1
Large	Openllet	9/9	6/9 (66.7%)	310732.4
Large	JFact	9/9	6/9 (66.7%)	334343.0
Large	Pellet	9/9	6/9 (66.7%)	337355.7

This companion view leaves the Standard conclusion unchanged, where ELK remains strongest at suite level, but its main purpose here is to reinforce the large-suite picture: once timeout penalties are incorporated, SPACL remains strongest on the large biomedical panel.

6.7.4. Supplementary Biomedical Extension: PATO

A supplementary matched three-run panel on `pato.owl` is reported in Table 11 and follows the same qualitative direction as the primary large suite. SPACL is the lowest successful median (3238 ms), ELK and HermiT remain successful but slower (6664 ms and 12362 ms), Pellet succeeds but is much slower (745159 ms), and Openllet/JFact time out on all three runs. We therefore use PATO as complementary biomedical evidence rather than as a replacement for the primary three-ontology panel.

6.7.5. External Validation: OWL2Bench Core Profiles

Table 12 reports repeated-run aggregate results on the OWL2Bench core profile ontologies.

SPACL completes all 12 runs and yields the smallest median wall-clock values among successful comparable runs in this suite. ELK, HermiT, Openllet, and Pellet also complete every run; JFact completes only the QL case under the stated timeout policy.

Table 11: Supplementary PATO Biomedical Panel (3 protocol-matched runs; timeout 900s)

Reasoner	Success / 3	Median Successful Wall (ms)	Status Note
SPACL	3/3	**3238**	all runs successful
ELK	3/3	6664	all runs successful
HermiT	3/3	12362	all runs successful
Pellet	3/3	745159	all runs successful
JFact	0/3	TO	timed out on all runs
Openllet	0/3	TO	timed out on all runs

Table 12: OWL2Bench Core Profile Suite Aggregate (3 clean runs; timeout 300s)

Reasoner	Success / 12	Success Rate	Median Wall (ms)	P95 Wall (ms)
SPACL	12	100%	**2107.5**	**2707**
Pellet	12	100%	2886.0	3135
ELK	12	100%	2999.0	3370
Openllet	12	100%	3204.5	3836
HermiT	12	100%	3573.0	4505
JFact	3	25%	3507	3762

6.7.6. Supplementary Deployment-Level Konclude Validation

Konclude was rerun separately under a preprocessing-enabled external-binary path that converts inputs to OWL/XML and merges imports before invocation. Under that supplementary deployment path, Konclude completed the Standard, Large, and OWL2Bench-core reruns, with aggregate median wall-clock values of 5285 ms (Standard), 34684 ms (Large), and 5382.5 ms (OWL2Bench core). These values are not mixed into the primary comparison tables because the invocation path differs from the main campaign. We therefore treat Konclude as deployment-level supplementary evidence rather than as a directly pooled comparator.

6.8. Nogood Learning Effectiveness

Because the primary head-to-head suites are mostly consistent ontologies, the benefit of nogood reuse is not fully visible in the main end-to-end tables. We therefore include the dedicated inconsistent-workload ablation in Table 13, which isolates the effect of enabling nogood learning under the same speculative runtime.

The inconsistent disjunctive case is the one that matters here: runtime drops by about 24%, roughly 28% of branches are pruned, and most repeated nogood checks stay on the local cache path. On consistent and hierarchy-heavy workloads,

Table 13: Nogood Learning Effectiveness (mean ± std, $n = 50$)

Ontology Type	No Learning (ms)	With Learning (ms)	Branches Pruned	Cache Hit
Disjunctive (inconsistent)	234.5 ± 12.3	178.3 ± 9.7	28.4%	82.3%
Disjunctive (consistent)	89.2 ± 5.1	87.1 ± 4.8	3.2%	91.7%
Hierarchical	45.6 ± 2.3	45.1 ± 2.1	0.1%	95.2%

nogood reuse has little effect, which is consistent with the main benchmark suites being dominated by consistent ontologies.

6.9. Limitations

Three limitations matter most for interpreting the results. First, the primary comparisons are deployment-level wall-clock measurements, so startup, parsing, and runtime-environment effects are part of the reported numbers. Second, the comparison set spans different logic-profile coverage, especially for ELK. Third, the current head-to-head suites are dominated by consistent ontologies, so nogood-learning gains are best read from the dedicated inconsistent-workload ablation rather than from the primary benchmark tables.

7. Conclusion and Future Work

SPACL combines speculative work stealing, verified nogood reuse, and a structural RDF/XML pipeline in one open-source implementation for the evaluated $\mathcal{ALC}/\mathcal{SHOIQ}$handling path. The empirical picture is mixed but coherent, and the paper should be read that way. On synthetic disjunctive workloads, the adaptive gate avoids pathological always-parallel overhead and permits net speedup once branch structure becomes large enough. On the main real-world biomedical panel (DOID, GO-Basic, UBERON), however, the dominant gain in the current evaluation comes from parser/materialization acceleration: stage-level A/B runs show about 97% parse-only reduction, and SPACL yields the smallest successful median end-to-end wall-clock values on all three ontologies in the primary large-suite campaign. The OWL2Bench core-profile suite shows a consistent deployment-level direction, with SPACL completing all 12 runs and yielding the smallest successful median wall-clock values on the four tested profiles in this harness. The resulting scientific takeaway is that, in this evaluated setting, end-to-end OWL reasoning performance is determined jointly by branch-management policy and ontology-ingest design, with the ingest side dominating the largest real-world gains.

These conclusions remain bounded by the current prototype and protocol. The main comparisons are single-machine, repeated-run wall-clock results un-

der a containerized CLI harness; they do not imply identical logic-profile coverage across all competitors, nor do they replace matched kernel-level reasoning measurements. The primary practical result in the current evaluation is parser- and materialization-side acceleration on large RDF/XML ontologies, while the speculative-reasoning contribution is best supported by the branch-heavy synthetic and inconsistent-workload evidence. PATO, ChEBI, and Konclude are therefore reported as supplementary evidence rather than folded into the primary comparison basis. Future work is concrete in three directions: (1) expand to broader inconsistent real-world corpora and heavier ABox-centered workloads, (2) run cleaner deployment-neutral comparator campaigns with explicit variance/significance reporting, and (3) strengthen scheduler-focused ablations that isolate adaptive gating and nogood reuse under fixed ingest conditions. SPACL is available at https://github.com/anusornc/SPACL.

Acknowledgments

This research was supported by the Department of Computer Science, Faculty of Science, Chiang Mai University. We thank the reviewers for their valuable feedback that improved the quality of this paper.

References

[1] B. Motik, P. F. Patel-Schneider, B. Parsia, C. Bock, A. Fokoue, P. Haase, R. Hoekstra, I. Horrocks, A. Ruttenberg, U. Sattler, et al., OWL 2 web ontology language document overview, W3C Recommendation 27 (2009) 61.
URL https://www.w3.org/TR/owl2-overview/

[2] F. Baader, D. Calvanese, D. L. McGuinness, D. Nardi, P. F. Patel-Schneider (Eds.), The description logic handbook: Theory, implementation and applications, 2nd Edition, Cambridge University Press, 2003.
URL https://www.cambridge.org/core/books/description-logic-handbook/

[3] E. Sirin, B. Parsia, B. Cuenca Grau, A. Kalyanpur, Y. Katz, Pellet: A practical OWL-DL reasoner, Journal of Web Semantics 5 (2) (2007) 51–53. doi:10.1016/j.websem.2007.03.004.

[4] R. S. Gonçalves, E. Paslaru Bontas, T. Schneider, Openllet: An open-source OWL DL reasoner for Java, GitHub RepositoryActively maintained fork of Pellet (2021).
URL https://github.com/GalacticOrganizer/openllet

[5] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, Z. Wang, Hermit: An OWL 2 reasoner, Journal of Automated Reasoning 53 (3) (2014) 245–269. doi:10.1007/s10817-014-9305-1.

[6] A. Steigmiller, T. Liebig, B. Glimm, Konclude: System description, Journal of Web Semantics 27 (2014) 78–85. doi:10.1016/j.websem.2014.06.002.

[7] B. Motik, R. Shearer, I. Horrocks, Hypertableau reasoning for description logics, Journal of Artificial Intelligence Research 36 (2009) 165–228. doi:10.1613/jair.2811.

[8] B. Parsia, N. Matentzoglu, R. S. Gonçalves, B. Glimm, A. Steigmiller, The owl reasoner evaluation (ore) 2015 competition report, ORE Workshop (2015).
URL http://ceur-ws.org/Vol-1387/

[9] Y. Kazakov, M. Krötzsch, F. Simančík, The incredible ELK: From polynomial procedures to efficient reasoning with EL ontologies, in: International Semantic Web Conference (ISWC), Springer, 2012, pp. 1–16. doi:10.1007/978-3-642-35176-1_1.

[10] Z. Quan, V. Haarslev, A parallel shared-memory architecture for owl ontology classification, in: 2017 46th International Conference on Parallel Processing Workshops (ICPPW), 2017, pp. 213–222. doi:10.1109/ICPPW.2017.38.

[11] Z. Quan, V. Haarslev, A framework for parallelizing owl classification in description logic reasoners, arXiv preprint arXiv:1906.07749 (2019).
URL https://arxiv.org/abs/1906.07749

[12] N. S. Arora, R. D. Blumofe, C. G. Plaxton, Thread scheduling for multiprogramming multiprocessors, in: ACM Symposium on Parallelism in Algorithms and Architectures (SPAA), 1998, pp. 119–129. doi:10.1145/277651.277678.

[13] A. Steigmiller, B. Glimm, Parallelised abox reasoning and query answering with expressive description logics (extended abstract), Description Logics

Workshop (extended abstract) (2020).
URL https://dblp.org/rec/conf/dlog/SteigmillerG20.html

[14] Y. Liu, P. McBrien, Spowl: Spark-based owl 2 reasoning materialisation, Proceedings of the 2nd International Workshop on Semantics-driven Big Data Analytics (SeBiDa), article no. 4 (2017). doi:10.1145/3070607.3070609.

[15] R. Gu, S. Wang, F. Wang, C. Yuan, Y. Huang, Cichlid: Efficient large scale rdfs/owl reasoning with spark, in: 2015 IEEE International Parallel and Distributed Processing Symposium, 2015, pp. 700–709. doi:10.1109/IPDPS.2015.46.

[16] A. Benítez-Hidalgo, J. García-Nieto, A. J. Nebro, J. F. Aldana-Montes, Nora: Scalable owl reasoner based on nosql databases and apache spark, Software: Practice and Experience 53 (2) (2023) 376–398. doi:10.1002/spe.3135.

[17] J. P. Marques-Silva, K. A. Sakallah, Grasp: A search algorithm for propositional satisfiability, IEEE Transactions on Computers 48 (5) (1999) 506–521. doi:10.1109/12.769433.

[18] G. Glorian, J.-M. Lagniez, C. Lecoutre, Nacre - a nogood and clause reasoning engine, in: Proceedings of the 6th International Workshop on Pragmatics of SAT (POS), Vol. 75 of EPiC Series in Computing, 2020, pp. 1–14. doi:10.29007/DXNB.

[19] W. Song, B. Spencer, W. Du, Complete classification of complex alcho ontologies using a hybrid reasoning approach, in: Description Logics Workshop, 2013, pp. 11–18.
URL http://ceur-ws.org/Vol-1014/paper_72.pdf

[20] B. Glimm, A. Steigmiller, T. Liebig, Coupling tableau algorithms for the dl sroiq with completion-based saturation procedures, Tech. rep., Ulm University (2014). doi:10.18725/OPARU-3211.

[21] Crossbeam Contributors, Crossbeam: Tools for concurrent programming in Rust, https://github.com/crossbeam-rs/crossbeam, accessed: 2026-02-02 (2023).

[22] P. L. Whetzel, N. F. Noy, N. H. Shah, P. R. Alexander, C. Nyulas, T. Tudorache, M. A. Musen, BioPortal: A web repository for biomedical ontologies,

Journal of Biomedical Semantics 4 (1) (2011) 1–8. doi:10.1186/2041-1480-4-1.